

UNIVERSIDAD NACIONAL MAYOR DE SAN MARCOS

Universidad del Perú, Decana de América



FACULTAD DE INGENIERÍA DE SISTEMAS E INFORMÁTICA

ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS

---

## **Semana 2 - POO**

---

**Curso:**

Programación de Computadoras II

**Docente:**

Javier Elmer Cabrera Díaz

**Autor:**

Mamani Huahualuque, Airton Abedal

**Lima, 8 de Septiembre del 2026**

# Índice

|                                                                              |          |
|------------------------------------------------------------------------------|----------|
| <b>1. Introducción</b>                                                       | <b>2</b> |
| <b>2. Desarrollo y Código Fuente</b>                                         | <b>2</b> |
| 2.1. Tarea 2.1: Clase Rectangulo y Encapsulamiento . . . . .                 | 2        |
| 2.2. Tarea 2.2: Relaciones de Asociación, Agregación y Composición . . . . . | 4        |
| <b>3. Conclusiones</b>                                                       | <b>9</b> |

# 1 Introducción

El paradigma de la Programación Orientada a Objetos (POO) constituye la columna vertebral del desarrollo de software moderno, permitiendo abstraer entidades complejas del mundo real hacia estructuras de código modulares, mantenibles y reutilizables. El presente informe aborda el desarrollo práctico de dos tareas fundamentales en el aprendizaje del lenguaje Java: la implementación del principio de encapsulamiento mediante una clase geométrica (Tarea 2.1) y el modelado de diversos niveles de asociación entre objetos (Tarea 2.2).

El objetivo principal de esta entrega es demostrar el dominio técnico en la construcción de clases robustas que protejan su estado interno mediante modificadores de acceso, métodos accesorios/modificadores y validaciones de dominio. Asimismo, se ilustran de forma gráfica y conceptual los tres tipos de relaciones estructurales entre clases: **Asociación Simple**, **Agregación** y **Composición**, destacando las diferencias de acoplamiento y ciclo de vida de los componentes en cada caso.

## 2 Desarrollo y Código Fuente

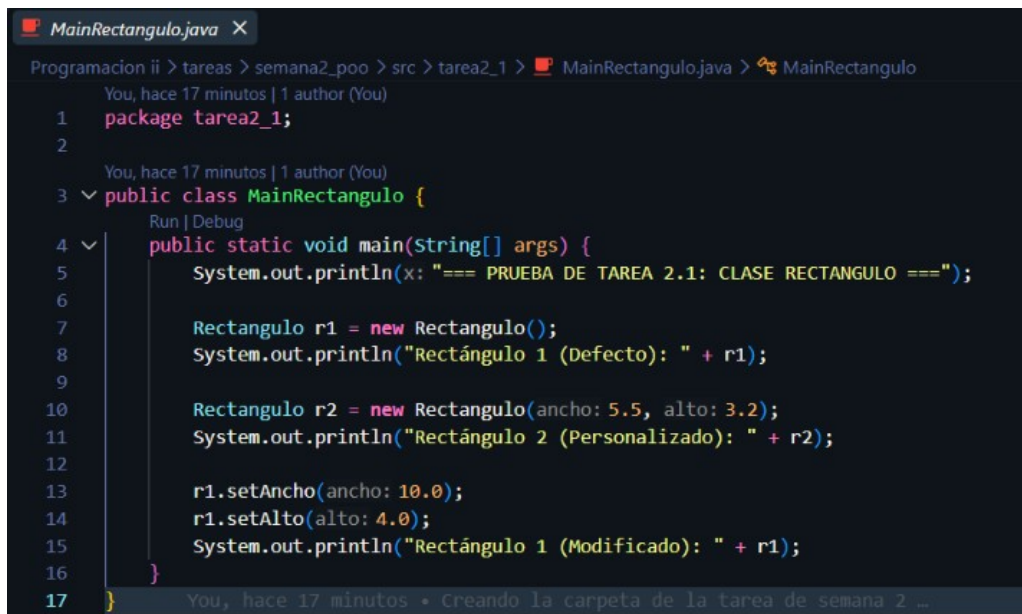
### 2.1 Tarea 2.1: Clase Rectangulo y Encapsulamiento

Para la Tarea 2.1, se diseñó la clase Rectangulo aplicando encapsulamiento estricto. Los atributos `ancho` y `alto` se definieron con visibilidad privada, forzando a que cualquier modificación se realice a través de sus respectivos métodos *setter*, los cuales garantizan valores mayores a cero. Se implementaron dos constructores (por defecto y parametrizado) y métodos para el cálculo de área y perímetro.

```
1 package tarea2_1;
2
3 public class Rectangulo {
4     private double ancho;
5     private double alto;
6
7     // Constructor por defecto
8     public Rectangulo() {
9         this.ancho = 1.0;
10        this.alto = 1.0;
11    }
12
13    // Constructor parametrizado
14    public Rectangulo(double ancho, double alto) {
15        setAncho(ancho);
16        setAlto(alto);
17    }
18
19    // Getters y Setters
20    public double getAncho() {
21        return ancho;
22    }
23
24    public void setAncho(double ancho) {
25        if (ancho > 0) {
26            this.ancho = ancho;
27        } else {
28            System.out.println("El ancho debe ser mayor que 0. Se asignó 1.0 por defecto.");
29            this.ancho = 1.0;
30        }
31    }
32
33    public double getAlto() {
34        return alto;
35    }
36
37    public void setAlto(double alto) {
38        if (alto > 0) {
39            this.alto = alto;
40        } else {
41            System.out.println("El alto debe ser mayor que 0. Se asignó 1.0 por defecto.");
42            this.alto = 1.0;
43        }
44    }
45
46    // Métodos de cálculo
47    public double calcularArea() {
48        return this.ancho * this.alto;
49    }
50
51    public double calcularPerimetro() {
52        return 2 * (this.ancho + this.alto);
53    }
54
55    @Override
56    public String toString() {
57        return "Rectangulo [Ancho = " + ancho + ", Alto = " + alto +
58            ", Área = " + calcularArea() + ", Perimetro = " + calcularPerimetro() + "];"
59    }
60 }
```

Figura 1: Código fuente de la clase Rectangulo.java.

**Nota:** Implementación de atributos privados con validación en setters y sobrescritura de toString().



```
1 package tarea2_1;
2
3 public class MainRectangulo {
4     public static void main(String[] args) {
5         System.out.println(x: "=== PRUEBA DE TAREA 2.1: CLASE RECTANGULO ===");
6
7         Rectangulo r1 = new Rectangulo();
8         System.out.println("Rectángulo 1 (Defecto): " + r1);
9
10        Rectangulo r2 = new Rectangulo(ancho: 5.5, alto: 3.2);
11        System.out.println("Rectángulo 2 (Personalizado): " + r2);
12
13        r1.setAncho(ancho: 10.0);
14        r1.setAlto(alto: 4.0);
15        System.out.println("Rectángulo 1 (Modificado): " + r1);
16    }
17 }
```

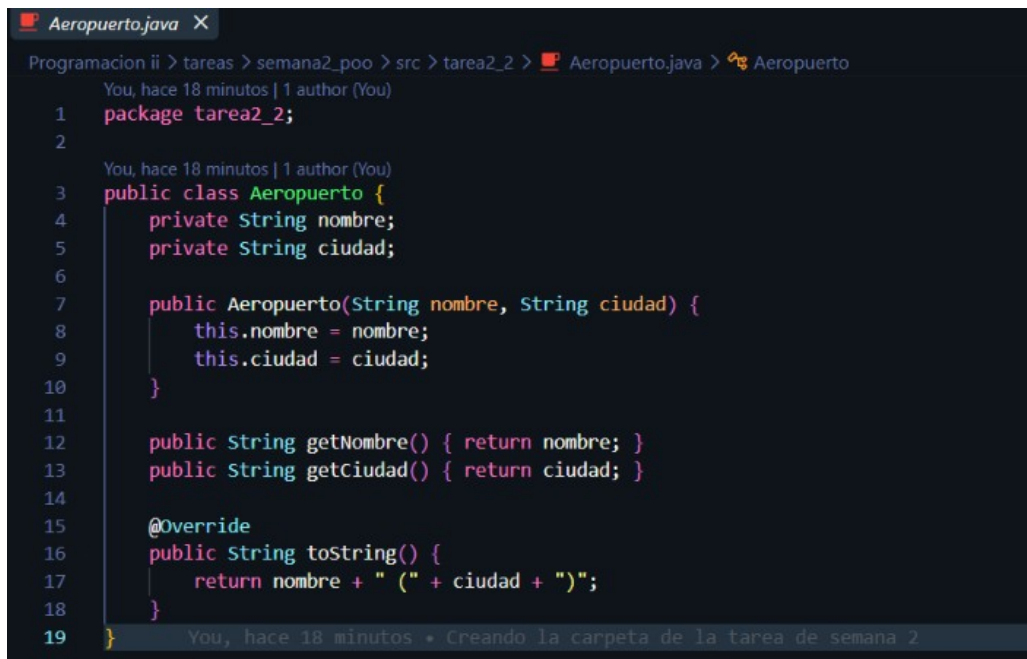
Figura 2: Clase de prueba MainRectangulo.java.

**Nota:** Métodos de prueba para verificar la creación de objetos por defecto, parametrizados y modificación de atributos.

## 2.2 Tarea 2.2: Relaciones de Asociación, Agregación y Composición

En la Tarea 2.2 se representan las tres variantes de interacción entre objetos en Java:

- **Asociación Simple (Vuelo - Aeropuerto):** Los aeropuertos existen de forma totalmente independiente del vuelo. Un vuelo referencia un aeropuerto de origen y uno de destino sin ser dueño de su ciclo de vida.
- **Agregación (Almacén - Montacargas):** Relación 'tiene-un' con acoplamiento débil. Un almacén utiliza un montacargas, pero si el almacén se destruye, el montacargas puede reasignarse a otro almacén.
- **Composición (Póliza - Cobertura):** Relación 'parte-de' con acoplamiento fuerte. La cobertura se instancia internamente dentro de la póliza de seguro. Si la póliza deja de existir, su cobertura asociada también se destruye.



```
Aeropuerto.java X
Programacion ii > tareas > semana2_poo > src > tarea2_2 > Aeropuerto.java > Aeropuerto
You, hace 18 minutos | 1 author (You)
1 package tarea2_2;
2
3 You, hace 18 minutos | 1 author (You)
4 public class Aeropuerto {
5     private String nombre;
6     private String ciudad;
7
8     public Aeropuerto(String nombre, String ciudad) {
9         this.nombre = nombre;
10        this.ciudad = ciudad;
11    }
12
13    public String getNombre() { return nombre; }
14    public String getCiudad() { return ciudad; }
15
16    @Override
17    public String toString() {
18        return nombre + " (" + ciudad + ")";
19    }
19 }
```

Figura 3: Clase Aeropuerto.java (Asociación Simple).

**Nota:** Definición de la entidad independiente Aeropuerto con atributos de nombre y ciudad.



```
Vuelo.java X
Programacion ii > tareas > semana2_poo > src > tarea2_2 > Vuelo.java > Vuelo
You, hace 18 minutos | 1 author (You)
1 package tarea2_2;
2
3 You, hace 18 minutos | 1 author (You)
4 public class Vuelo {
5     private String numeroVuelo;
6     private Aeropuerto origen;
7     private Aeropuerto destino;
8
9     public Vuelo(String numeroVuelo, Aeropuerto origen, Aeropuerto destino) {
10        this.numeroVuelo = numeroVuelo;
11        this.origen = origen;
12        this.destino = destino;
13    }
14
15    public void mostrarDetalle() {
16        System.out.println("Vuelo " + numeroVuelo + " | De: " + origen + " -> A: " + destino);
17    }
17 }
```

Figura 4: Clase Vuelo.java (Asociación Simple).

**Nota:** Incorporación de atributos de tipo Aeropuerto para relacionar origen y destino.

```

Montacargas.java X
Programación ii > tareas > semana2_poo > src > tarea2_2 > Montacargas.java > Montacargas
You, hace 18 minutos | 1 author (You)
1 package tarea2_2;
2
3 You, hace 18 minutos | 1 author (You)
4 public class Montacargas {
5     private String marca;
6     private double capacidadToneladas;
7
8     public Montacargas(String marca, double capacidadToneladas) {
9         this.marca = marca;
10        this.capacidadToneladas = capacidadToneladas;
11    }
12
13    @Override
14    public String toString() {
15        return "Montacargas " + marca + " [" + capacidadToneladas + " Tn]";
16    }
17 }
You, hace 18 minutos • Creando la carpeta de la tarea de semana 2 ...

```

Figura 5: Clase Montacargas.java (Agregación).

**Nota:** Representación del recurso externo Montacargas que puede operar de forma independiente.

```

Almacen.java X
Programación ii > tareas > semana2_poo > src > tarea2_2 > Almacen.java > Almacen
You, hace 19 minutos | 1 author (You)
1 package tarea2_2;
2
3 You, hace 19 minutos | 1 author (You)
4 public class Almacen {
5     private String nombre;
6     private Montacargas montacargas; // Agregación (el montacargas existe independientemente)
7
8     public Almacen(String nombre) {
9         this.nombre = nombre;
10    }
11
12    public void asignarMontacargas(Montacargas m) {
13        this.montacargas = m;
14    }
15
16    public void mostrarEstado() {
17        System.out.println("Almacén: " + nombre + " | " +
18            (montacargas != null ? "Asignado: " + montacargas : "Sin montacargas asignado"));
19    }
20 }
You, hace 19 minutos • Creando la carpeta de la tarea de semana 2 ...

```

Figura 6: Clase Almacen.java (Agregación).

**Nota:** Relación de agregación mediante la asignación externa de un objeto Montacargas.



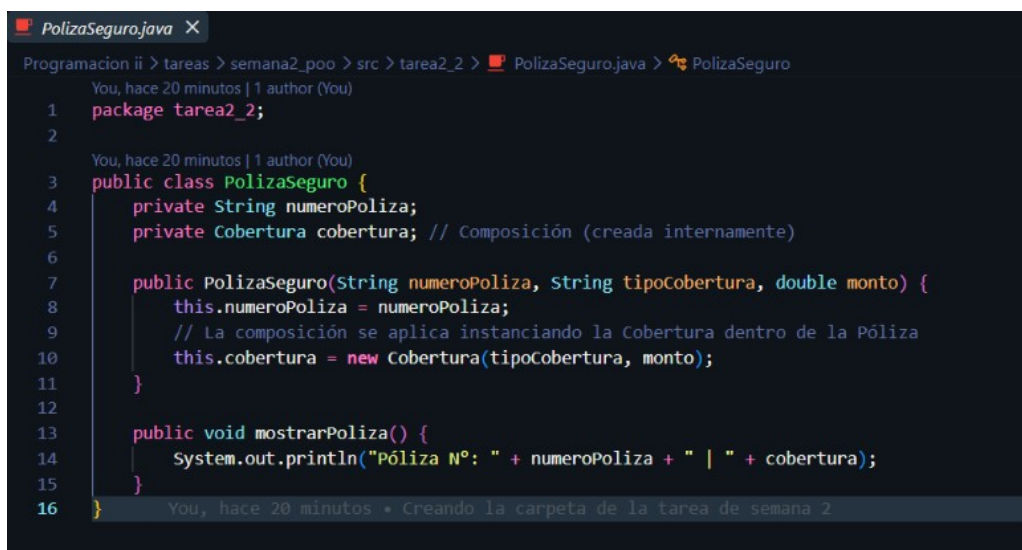
```

Cobertura.java X
Programacion ii > tareas > semana2_poo > src > tarea2_2 > Cobertura.java > Cobertura
You, hace 19 minutos | 1 author (You)
1 package tarea2_2;
2
3 You, hace 19 minutos | 1 author (You)
4 public class Cobertura {
5     private String tipo;
6     private double montoMaximo;
7
8     public Cobertura(String tipo, double montoMaximo) {
9         this.tipo = tipo;
10        this.montoMaximo = montoMaximo;
11    }
12
13    @Override
14    public String toString() {
15        return "Cobertura: " + tipo + " (Hasta S/ " + montoMaximo + ")";
16    }
17 }
You, hace 19 minutos • Creando la carpeta de la tarea de semana 2

```

Figura 7: Clase Cobertura.java (Composición).

**Nota:** Clase dependiente que representa las características de una cobertura de seguro.



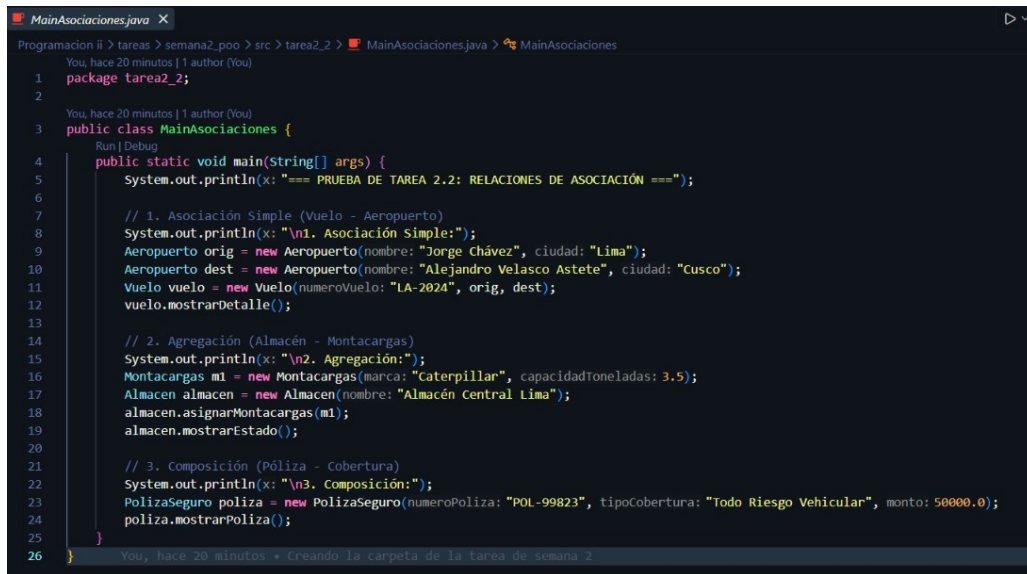
```

PolizaSeguro.java X
Programacion ii > tareas > semana2_poo > src > tarea2_2 > PolizaSeguro.java > PolizaSeguro
You, hace 20 minutos | 1 author (You)
1 package tarea2_2;
2
3 You, hace 20 minutos | 1 author (You)
4 public class PolizaSeguro {
5     private String numeroPoliza;
6     private Cobertura cobertura; // Composición (creada internamente)
7
8     public PolizaSeguro(String numeroPoliza, String tipoCobertura, double monto) {
9         this.numeroPoliza = numeroPoliza;
10        // La composición se aplica instanciando la Cobertura dentro de la Póliza
11        this.cobertura = new Cobertura(tipoCobertura, monto);
12    }
13
14    public void mostrarPoliza() {
15        System.out.println("Póliza N°: " + numeroPoliza + " | " + cobertura);
16    }
17 }
You, hace 20 minutos • Creando la carpeta de la tarea de semana 2

```

Figura 8: Clase PolizaSeguro.java (Composición).

**Nota:** Instanciación directa e interna del objeto Cobertura dentro del constructor de PolizaSeguro.



```
1 package tarea2_2;
2
3 public class MainAsociaciones {
4     public static void main(String[] args) {
5         System.out.println(x: "=== PRUEBA DE TAREA 2.2: RELACIONES DE ASOCIACIÓN ===");
6
7         // 1. Asociación Simple (Vuelo - Aeropuerto)
8         System.out.println(x: "\n1. Asociación Simple:");
9         Aeropuerto orig = new Aeropuerto(nombre: "Jorge Chávez", ciudad: "Lima");
10        Aeropuerto dest = new Aeropuerto(nombre: "Alejandro Velasco Astete", ciudad: "Cusco");
11        Vuelo vuelo = new Vuelo(numeroVuelo: "LA-2024", orig, dest);
12        vuelo.mostrarDetalle();
13
14        // 2. Agregación (Almacén - Montacargas)
15        System.out.println(x: "\n2. Agregación:");
16        Montacargas m1 = new Montacargas(marca: "Caterpillar", capacidadToneladas: 3.5);
17        Almacen almacen = new Almacen(nombre: "Almacén Central Lima");
18        almacen.asignarMontacargas(m1);
19        almacen.mostrarEstado();
20
21        // 3. Composición (Póliza - Cobertura)
22        System.out.println(x: "\n3. Composición:");
23        PolizaSeguro poliza = new PolizaSeguro(numeroPoliza: "POL-99823", tipoCobertura: "Todo Riesgo Vehicular", monto: 50000.0);
24        poliza.mostrarPoliza();
25    }
26 }
```

Figura 9: Ejecución principal MainAsociaciones.java.

**Nota:** Programa principal que ejecuta las tres relaciones entre objetos y muestra el resultado en la consola.

## 3 Conclusiones

- **Garantía de Integridad mediante Encapsulamiento:** La restricción del acceso directo a los atributos de un objeto a través de modificadores `private` e interfaces de acceso (*getters* y *setters*) asegura que los datos no puedan tomar estados inválidos o inconsistentes dentro del sistema.
- **Flexibilidad y Reutilización de Código:** Al comprender la diferencia entre Agregación y Composición, el desarrollador puede tomar decisiones de arquitectura adecuadas: optar por agregación cuando se requiera compartir objetos entre distintos módulos, y composición cuando la existencia de un objeto secundario dependa indefectiblemente del objeto contenedor.
- **Mantenibilidad y Diseño Limpio:** El uso de la anotación `@Override` para rediseñar métodos estándar como `toString()` simplifica significativamente el proceso de depuración y salida de datos, promoviendo un código legible, autocontenido y estándar bajo las mejores prácticas de la industria Java.